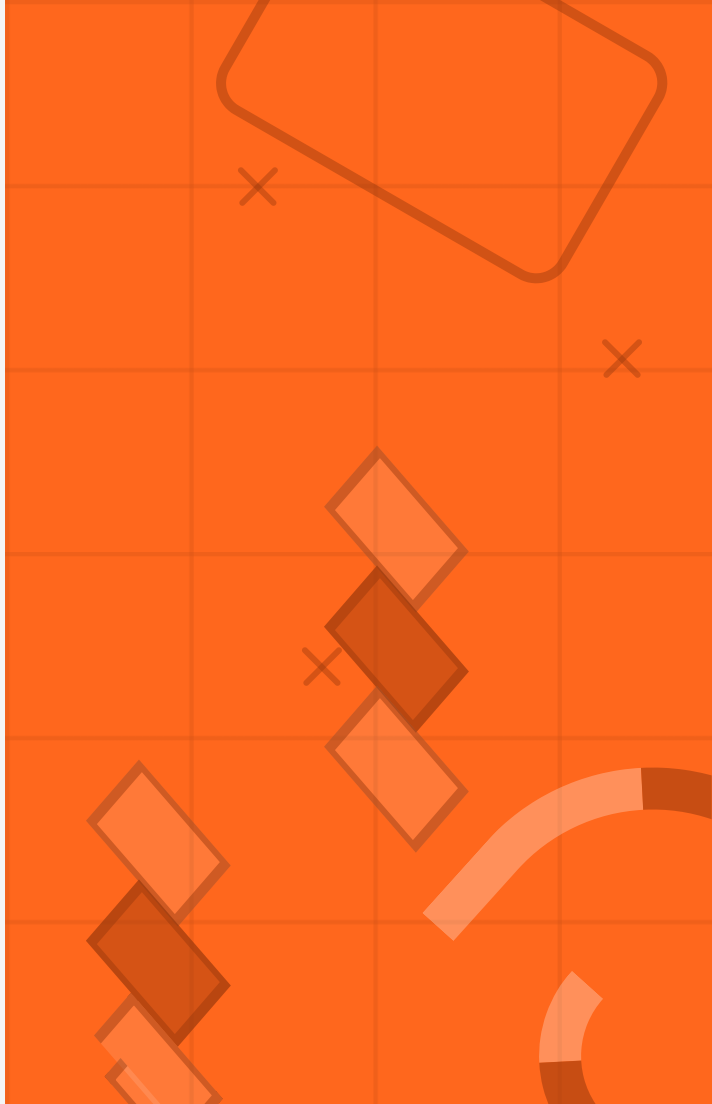
Grafana Labs

How Grafana helps you build faster

The shortest path to a better project is a tighter feedback loop: see what happened, fix the right thing, and move on.

Jo Guerreiro

Grafana Labs · RAISE hackathon ::



Build speed is feedback-loop speed

Can your team answer a **new question** without editing code?

Monitoring, observability, and reliability

Monitoring

Question: Is it broken?

Best for known thresholds and expected failure modes

Observability

Question: Why did it behave that way?

Best when a failure shows up in a way you did not predict

Reliability: the loop around both

- Turns those signals into alerts, ownership, and fixes
- Detect → investigate → decide → repair → verify
- Same loop whether the responder is you or an agent

> In plain English: you can investigate from the outside instead of adding another `printf` and running it again.

Four kinds of evidence

Logs

- What happened?
- Errors, tool output, model responses, request context

Traces

- Where did it go?
- One request across API, model, tools, database, and UI

Metrics

- How much?
- Latency, errors, tokens, queue depth, throughput

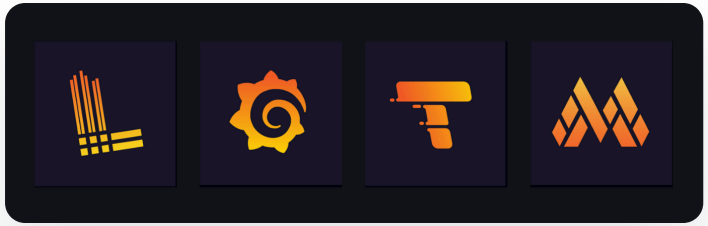
Profiles

- What used resources?
- CPU, memory, allocations, and hot paths over time

What the LGTM stack actually does

Store each signal in the shape it wants

- **Loki** for logs
- **Tempo** for traces
- **Mimir** for Prometheus-compatible metrics
- **Grafana** as the shared place to inspect and act



Grafana ties the signals together

1. A signal moves
2. You ask the next question
3. You pivot to the related evidence
4. Someone fixes the thing that matters

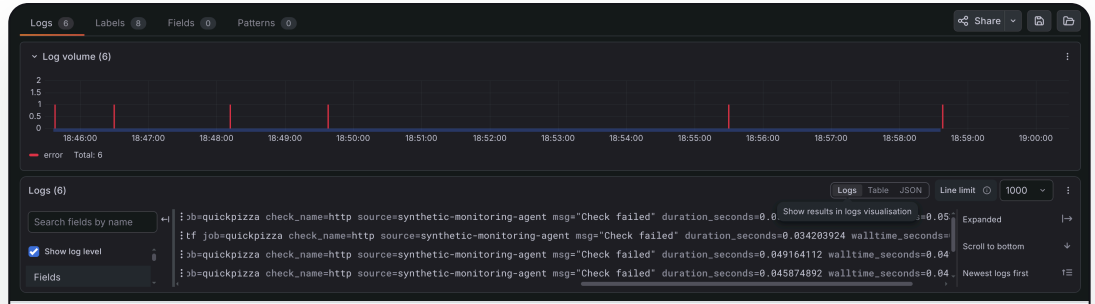
The win is less time guessing.

Loki

Searchable logs without indexing the whole book

- Use labels for the fields you filter on often: `app`, `route`, `env`, `tenant`
- Keep detail with many unique values in the log body
- Shared labels are also how you pivot from a trace span back to these logs
- Good logs make it easy to reconstruct the event around a failure

```
{app="ai-project", route="/chat", env="hackathon", tenant="team-a"}
tool_call failed: calendar_api timeout after 2s
```



Grafana · Loki drilldown

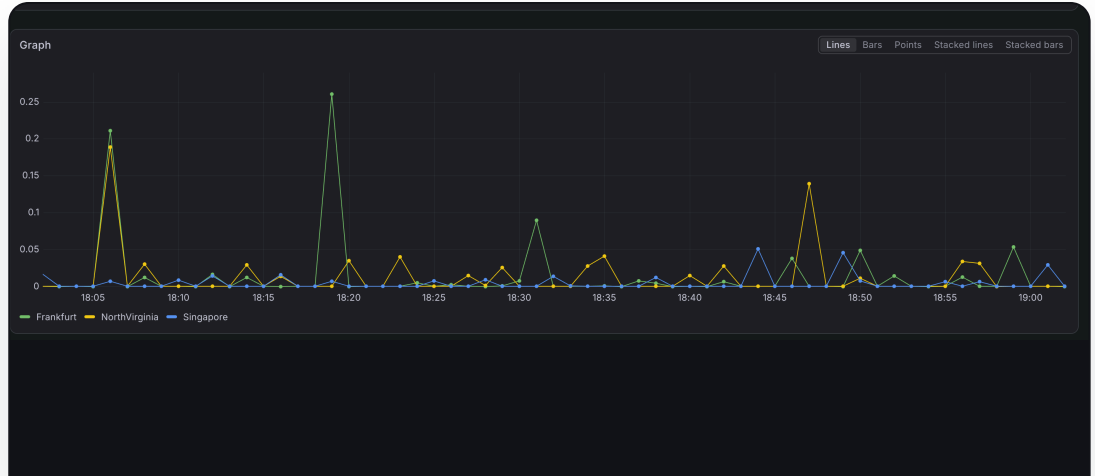
Prometheus + Mimir

Turn behavior
into numbers
you can alert on

- Metrics tell you **where** to look and **when** user impact changed
- The query language lets you slice by route, model, tenant, region, or job
- Mimir gives you longer retention, more scale, and room for many teams to share one system

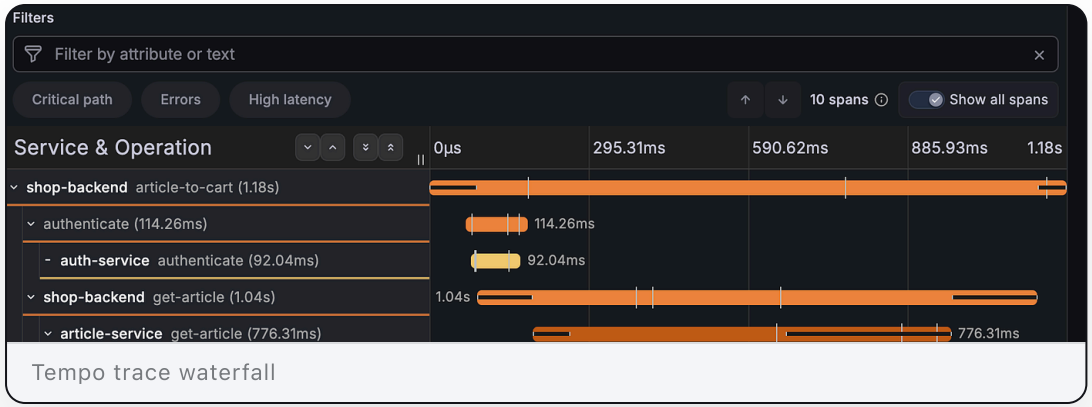
```
sum(rate(http_requests_total{app="ai-project", status=~"5.."}[5m]))
/
sum(rate(http_requests_total{app="ai-project"}[5m]))
```

Metrics rarely tell you why on their own. They tell you where to go next.



- Traces connect API work, model calls, tools, storage, and UI updates
- Trace links and shared labels let you jump from a slow span straight to the logs inside it
- In AI apps, traces show whether retrieval, tooling, auth, or the model burned the time

User prompt -> API -> Model -> Tool -> Storage



Tempo trace waterfall

Tempo

Follow one request across service boundaries

Alloy + OpenTelemetry are the plumbing

What they are

- **OpenTelemetry (OTel):** an open standard for logs, metrics, and traces that isn't tied to one vendor
- **Grafana Alloy:** the OTel Collector that receives, enriches, and routes that telemetry

Instrument one path well

1. Emit logs, metrics, and traces from the app
2. Send them through Alloy
3. Filter, enrich, and route them
4. Store them in Loki, Mimir, and Tempo

Instrument once. Ask many questions.

Quick start with your agent

Prompt this

1. "Add Grafana Alloy as a local OTel collector for this project"
2. "Instrument the API and the model calls with OpenTelemetry"
3. "Export traces, logs, and metrics through Alloy to Grafana Cloud"
4. "Run it, send a few requests, and check Grafana Cloud"

What the agent should produce

```
config.alloy
  otelcol.receiver.otlp  → listens on 4317
  otelcol.exporter.otlp  → ships to Grafana Cloud

app code
  OTel SDK + auto-instrumentation
  OTLP endpoint = localhost:4317
```

Same prompt, any language. The agent writes the plumbing; you just point it at Grafana Cloud.

Your classic grafana demo

gcx: Grafana in your terminal and agent

Why it matters

- Query logs, metrics, traces, profiles, alerts and dashboards
- Use the same Grafana data from your editor or CI
- Give coding agents a control plane, not copied screenshots

```
gcx logs query '{app="ai-project"} |= "error"' --since 30m
gcx metrics query 'rate(http_requests_total[5m])' --since 1h
gcx traces query '{.service.name="ai-project"}' --since 30m
```

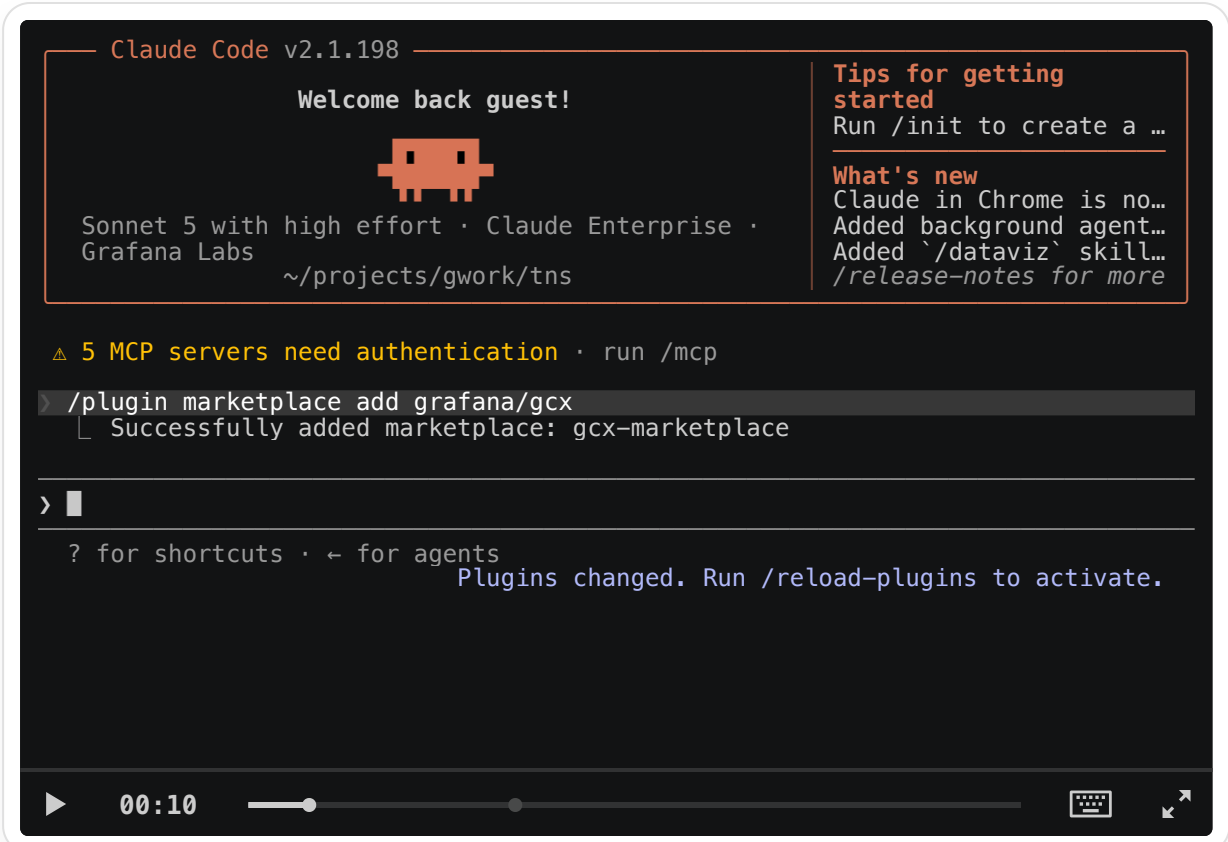
Claude Code plugin

```
/plugin marketplace add grafana/gcx
/plugin install gcx@gcx-marketplace
/setup-gcx
```

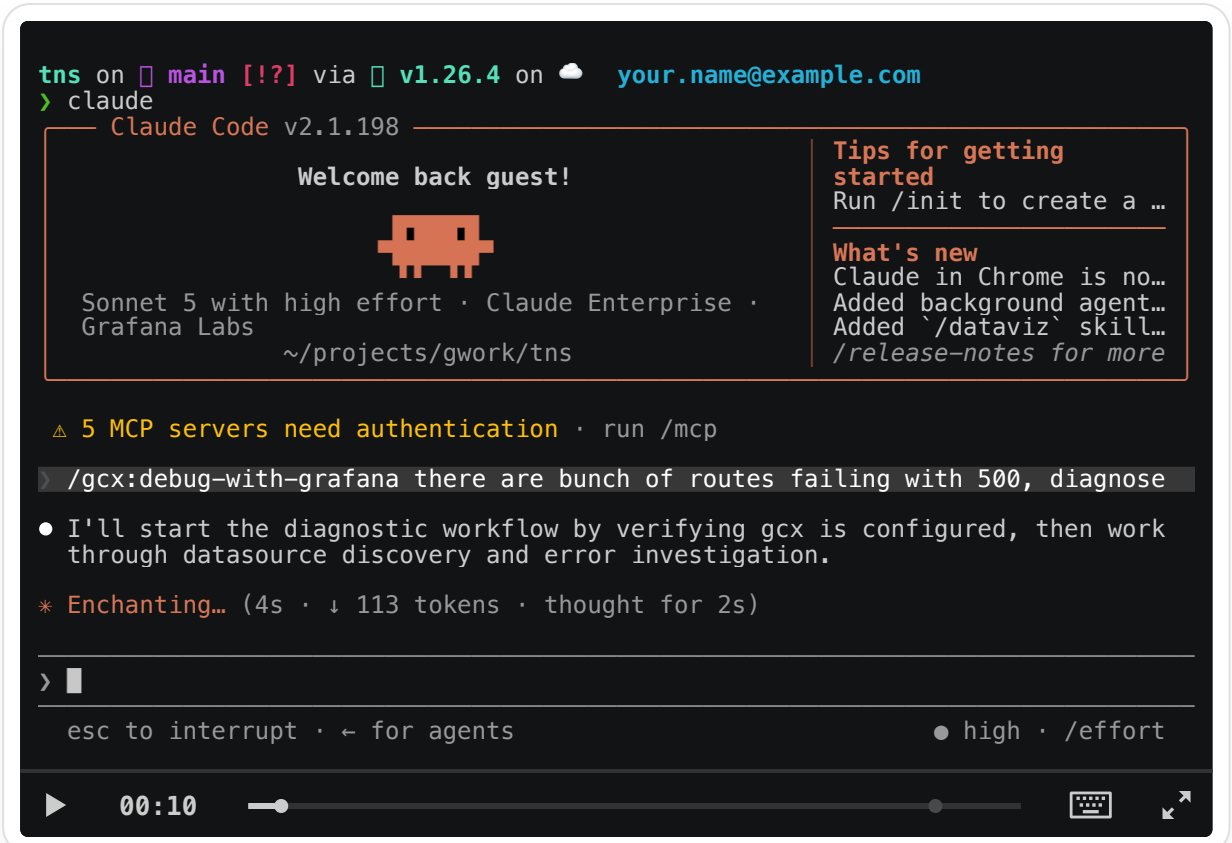
- Installs skills for setup, dashboards, exploring your data sources, alert investigation, SLOs, and debugging
- Also supports other `.agents`-compatible harnesses with `gcx agent skills install --all`

github.com/grafana/gcx

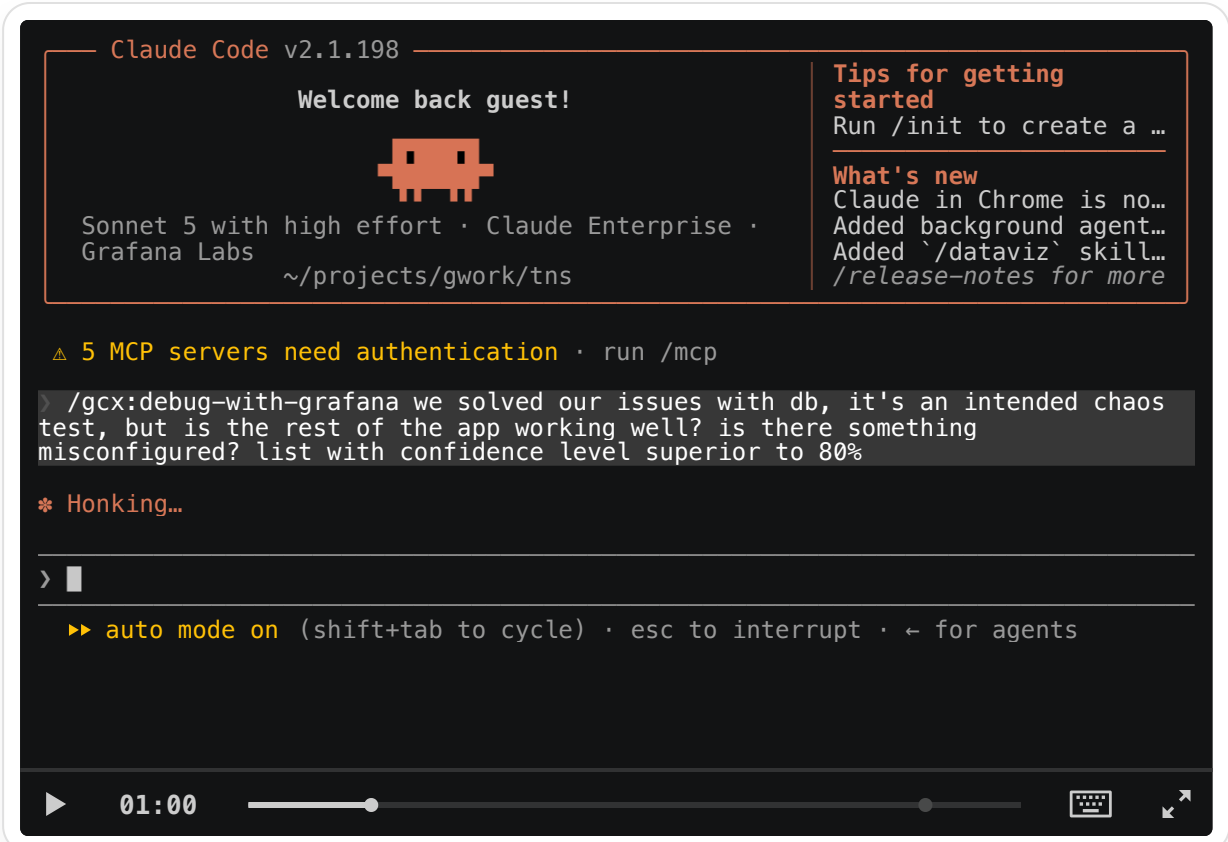
Setting up gcx



Trace-guided API fix



"Fix my project please"



Give your agent evidence, not vibes

Vibes

- "the intended flow failed"
- A pasted stack trace with no timestamp
- "it's slow," no request ID
- "worked the first time but not the second"

The agent has to guess what changed

Evidence

- The trace for the exact request: `/chat` → retrieve docs → call model → call tool
- Logs for the failing span
- Metrics before and after
- A dashboard or query it can rerun

An agent with evidence proposes a **scoped fix**. An agent with vibes, not so much.